

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №2
по дисциплине
«Алгоритмические основы компьютерной графики»

Студент,

группы 5130201/30101

_____ Мелещенко С. И.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Алгоритм отсечения отрезка выпуклым телом. Алгоритм Кируса-Бека	5
2.1 Отсекатель — девятигранник	9
2.1.1 Вычисление нормалей	9
2.1.2 Опорные точки	10
2.1.3 Алгоритм отсечения Кируса-Бека для девятигранника	10
2.1.4 Пример вычисления для одной грани	11
2.2 Проверка выпуклости	12
2.3 Реализация на языке C++	13
2.3.1 Класс Point3D	13
2.3.2 Структура грани (Face)	13
2.3.3 Вычисление внутренней нормали грани	14
2.3.4 Класс выпуклого многогранника (девятигранник)	14
2.3.5 Отрезок и алгоритм Кируса-Бека	15
2.3.6 Инициализация тестовых отрезков и главная функция	16
3 Результаты работы	18
4 Сравнение результатов работы алгоритмов	20
Заключение	22

Введение

В компьютерной графике и вычислительной геометрии задачи отсечения занимают центральное место. Основное применение алгоритмов отсечения - это отбор той информации, которая необходима для визуализации конкретной сцены или вида, как части более обширной обстановки. Но кроме этого отсечение применяется в алгоритмах удаления невидимых линий и поверхностей, при построении теней, а также при формировании фактуры.

Алгоритм Кируса-Бека является одним из наиболее эффективных и общих методов отсечения отрезков. Алгоритм Кируса-Бека может отсекал отрезки произвольным выпуклым многогранником, что делает его универсальным инструментом для трехмерной графики.

В данной работе рассматривается реализация трехмерного алгоритма Кируса-Бека для отсечения отрезков выпуклым телом (девятигранник), а также проводится сравнительный анализ производительности собственной реализации и векторизованной версии с использованием библиотеки NumPy.

1 Постановка задачи

Цель работы: реализовать алгоритм отсечения отрезков выпуклым телом методом Кируса-Бека в трёхмерном пространстве и провести сравнение производительности с библиотечной реализацией.

Задачи.

1. Изучить теоретические основы алгоритма Кируса-Бека.
2. Разработать структуры данных для представления геометрических примитивов: точки, векторы, грани, многогранники, отрезки.
3. Реализовать алгоритм отсечения на языке C++.
4. Провести тестирование корректности на примерах отрезков, заведомо пересекающих отсекатель.
5. Выполнить анализ производительности алгоритма.
6. Сравнить полученные результаты с библиотечной реализацией на Python.
7. Визуализировать исходные и отсечённые отрезки в трёхмерном пространстве.

Дано:

- Выпуклый многогранник-отсекатель.
- Множество отрезков в трёхмерном пространстве, произвольно ориентированных.

Требуется.

1. Определить видимую часть каждого отрезка, находящуюся внутри отсекателя.
2. Отобразить исходные и отсечённые отрезки графически (либо не отображать, если они изначально не пересекают девятигранник).
3. Измерить время работы алгоритма при разном количестве отрезков.

2 Алгоритм отсечения отрезка выпуклым телом. Алгоритм Кируса-Бека

Пусть отсекаТЕЛЬ — выпуклый многогранник (в трёхмерном пространстве), заданный набором граней. Каждая грань определяется:

- произвольной точкой $\mathbf{F}_i$, лежащей на плоскости грани;
- внутренней нормалью $\mathbf{n}_i$ (единичный вектор, направленный внутрь многогранника).

Отсекаемый отрезок задан двумя точками $\mathbf{P}_1$ и $\mathbf{P}_2$. Его параметрическое уравнение:

$$\mathbf{P}(t) = \mathbf{P}_1 + (\mathbf{P}_2 - \mathbf{P}_1)t, \quad 0 \leq t \leq 1.$$

Обозначим $\mathbf{D} = \mathbf{P}_2 - \mathbf{P}_1$ — направляющий вектор отрезка.

Условие пересечения с плоскостью грани. Точка $\mathbf{P}(t)$ принадлежит плоскости i -й грани тогда и только тогда, когда скалярное произведение вектора $\mathbf{P}(t) - \mathbf{F}_i$ на нормаль $\mathbf{n}_i$ равно нулю:

$$\mathbf{n}_i \cdot (\mathbf{P}(t) - \mathbf{F}_i) = 0. \quad (1)$$

Вывод параметра t_i . Подставим $\mathbf{P}(t)$ в (1):

$$\mathbf{n}_i \cdot (\mathbf{P}_1 + \mathbf{D}t - \mathbf{F}_i) = 0.$$

Раскрывая скобки, получаем:

$$\mathbf{n}_i \cdot (\mathbf{P}_1 - \mathbf{F}_i) + (\mathbf{n}_i \cdot \mathbf{D})t = 0.$$

Введём обозначение $\mathbf{w}_i = \mathbf{P}_1 - \mathbf{F}_i$. Тогда:

$$\mathbf{w}_i \cdot \mathbf{n}_i + (\mathbf{D} \cdot \mathbf{n}_i)t = 0.$$

Если $\mathbf{D} \cdot \mathbf{n}_i \neq 0$ (отрезок не параллелен грани), находим параметр пересечения:

$$t_i = -\frac{\mathbf{w}_i \cdot \mathbf{n}_i}{\mathbf{D} \cdot \mathbf{n}_i}. \quad (2)$$

Значение t_i может лежать вне интервала $[0, 1]$; это означает, что прямая пересекает плоскость грани за пределами отрезка.

Определение интервала видимости. Для выпуклого многогранника пересечение прямой с ним (если оно не пусто) представляет собой один отрезок. Идея алгоритма — последовательно «отсекать» от исходного отрезка части, лежащие снаружи, путём сужения интервала параметра t .

Инициализируем:

$$t_{\text{low}} = 0, \quad t_{\text{high}} = 1.$$

Для каждой грани i :

- Если $|\mathbf{D} \cdot \mathbf{n}_i| = 0$ (отрезок параллелен грани):
 - если $\mathbf{w}_i \cdot \mathbf{n}_i = 0$ — отрезок целиком снаружи, отбрасываем;
 - иначе — грань не ограничивает интервал.
- Иначе вычисляем t_i по формуле (2).
 - Если $\mathbf{D} \cdot \mathbf{n}_i > 0$ — отрезок движется внутрь относительно данной грани, пересечение даёт нижнюю границу:

$$t_{\text{low}} = \max(t_{\text{low}}, t_i).$$

- Если $\mathbf{D} \cdot \mathbf{n}_i < 0$ — отрезок движется наружу, пересечение даёт верхнюю границу:

$$t_{\text{high}} = \min(t_{\text{high}}, t_i).$$

После обработки всех граней приводим границы к допустимому диапазону:

$$t_{\text{low}} = \max(0, t_{\text{low}}), \quad t_{\text{high}} = \min(1, t_{\text{high}}).$$

Видимая часть отрезка есть $\mathbf{P}(t)$ при $t \in [t_{\text{low}}, t_{\text{high}}]$. Точки входа и выхода вычисляются как:

$$\mathbf{P}_{\text{in}} = \mathbf{P}_1 + t_{\text{low}}\mathbf{D}, \quad \mathbf{P}_{\text{out}} = \mathbf{P}_1 + t_{\text{high}}\mathbf{D}.$$

Таким образом, алгоритм сводится к линейному сканированию всех граней, вычислению одного параметра на грань и обновлению двух переменных. Его сложность $O(k)$, где k — число граней).

Пример 3.17. Трёхмерный алгоритм Кируса — Бека

На рис. 1 показан отрезок от $P_1(-2, -1, 1/2)$ до $P_2(3/2, 3/2, -1/2)$, который нужно отсечь по объёму с координатами $(x_l, x_r, y_d, y_u, z_f, z_c) = (-1, 1, -1, 1, 1, -1)$. Шесть внутренних нормалей к граням отсекающего, очевидно, равны:

$$\text{верхняя: } n_u = -j = [0 \ -1 \ 0]$$

$$\text{нижняя: } n_d = j = [0 \ 1 \ 0]$$

$$\text{правая: } n_r = -i = [-1 \ 0 \ 0]$$

$$\text{левая: } n_l = i = [1 \ 0 \ 0]$$

$$\text{ближняя: } n_c = -k = [0 \ 0 \ -1]$$

$$\text{дальняя: } n_f = k = [0 \ 0 \ 1]$$

Выбор точек, лежащих на каждой грани отсекающего, также очевиден. Достаточно взять две точки, лежащие на концах одной из главных диагоналей параллелепипеда. Итак,

$$f_u = f_r = f_c(1, 1, 1), \quad f_d = f_l = f_f(-1, -1, -1).$$

Вместо этих точек можно взять центры или любые угловые точки на соответствующих гранях.

Директриса отрезка P_1P_2 равна:

$$D = P_2 - P_1 = [3/2 \ 3/2 \ -1/2] - [-2 \ -1 \ 1/2] = [7/2 \ 5/2 \ -1].$$

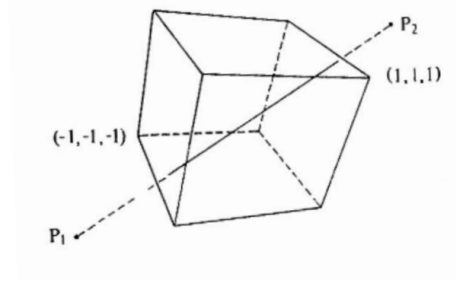


Рис. 1. Отсечение Кируса-Бека: трехмерный прямоугольный объем

Для граничной точки $f_l(-1, -1, -1)$:

$$w = P_1 - f = [-2 \ -1 \ 1/2] - [-1 \ -1 \ -1] = [-1 \ 0 \ 3/2]$$

а внутренняя нормаль к левой грани отсекающей равна

$$n_l = [1 \ 0 \ 0].$$

Следовательно,

$$D \cdot n_l = [7/2 \ 5/2 \ -1] \cdot [1 \ 0 \ 0] = 7/2 > 0$$

что соответствует нижнему пределу, а

$$w \cdot n_l = [-1 \ 0 \ 3/2] \cdot [1 \ 0 \ 0] = -1$$

и

$$t_l = -(-1)/(7/2) = 2/7.$$

Полные результаты работы алгоритма собраны в таблице ниже.

Грань	n	f	w	$w \cdot n$	$D \cdot n^{1)}$	t_H	t_B
Верхняя	$[0 \ -1 \ 0]$	$(1, 1, 1)$	$[-3 \ -2 \ -1/2]$	2	$-5/2$		$4/5$
Нижняя	$[0 \ 1 \ 0]$	$(-1, -1, -1)$	$[-1 \ 0 \ 3/2]$	0	$5/2$	0	
Правая	$[-1 \ 0 \ 0]$	$(1, 1, 1)$	$[-3 \ -2 \ -1/2]$	3	$-7/2$		$6/7$
Левая	$[1 \ 0 \ 0]$	$(-1, -1, -1)$	$[-1 \ 0 \ 3/2]$	-1	$7/2$	$2/7$	
Ближняя	$[0 \ 0 \ -1]$	$(1, 1, 1)$	$[-3 \ -2 \ -1/2]$	$1/2$	1	$-1/2$	
Дальняя	$[0 \ 0 \ 1]$	$(-1, -1, -1)$	$[-1 \ 0 \ 3/2]$	$3/2$	-1		$3/2$

¹⁾ При $D \cdot n < 0$, верхний предел (t_B); при $D \cdot n > 0$, нижний предел (t_H).

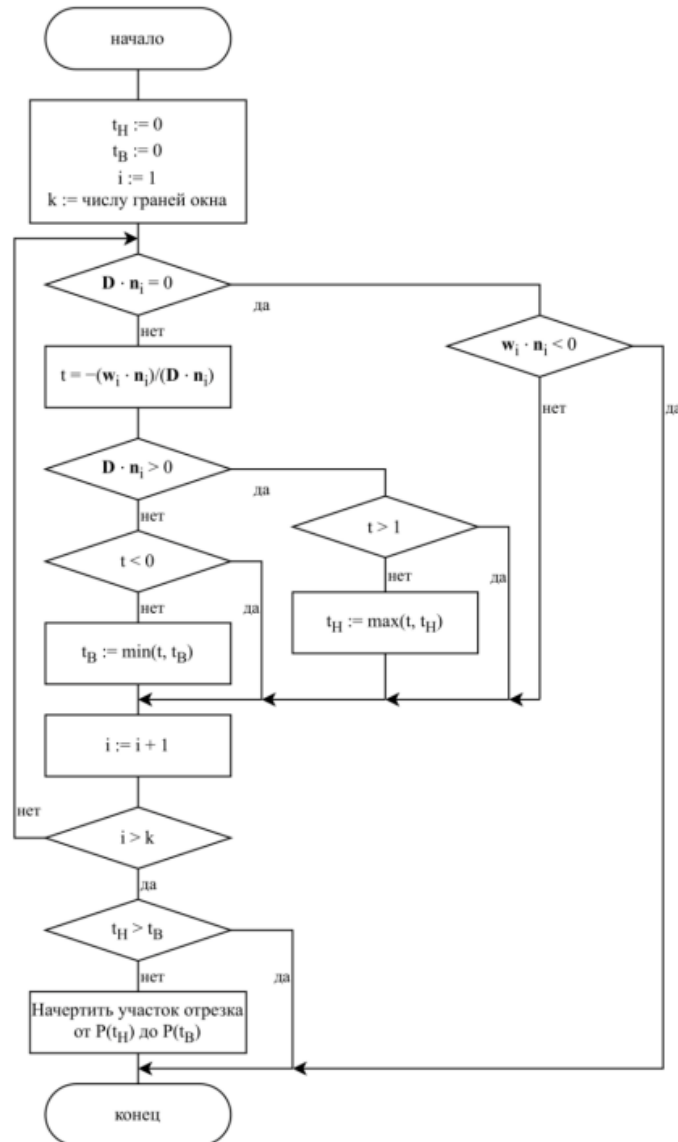
Анализ таблицы показывает, что максимальным из нижних значений будет $t_d = 2/7$, а минимальным из верхних значений будет $t_u = 4/5$. Параметрическое представление отрезка P_1P_2 имеет вид:

$$P(t) = [-2 \ -1 \ 1/2] + [7/2 \ 5/2 \ -1]t.$$

Подстановка сюда t_d и t_u дает:

$$P(2/7) = [-2 \ -1 \ 1/2] + [7/2 \ 5/2 \ -1] \cdot (2/7) = [-1 \ -2/7 \ 3/14],$$

$$P(4/5) = [-2 \ -1 \ 1/2] + [7/2 \ 5/2 \ -1] \cdot (4/5) = [4/5 \ 1 \ -3/10].$$



Сложность алгоритма Кируса-Бека: $O(k)$, где k — количество граней.

Рис. 2. Блок-схема алгоритма Кируса-Бека

2.1 Отсекатель — девятигранник

В качестве отсекателя используется многогранник с 9 треугольными гранями, построенный на 7 вершинах. Вершины выбраны так, чтобы многогранник был замкнуто выпуклым:

$$\begin{aligned}v_1 &= (0.0, 0.0, 0.0), & v_2 &= (3.0, 0.2, 0.1), & v_3 &= (2.5, 2.0, -0.2), \\v_4 &= (0.3, 1.8, 0.0), & v_5 &= (0.5, 0.4, 2.2), & v_6 &= (2.2, 0.1, 2.0), \\v_7 &= (1.5, 2.1, 1.9).\end{aligned}$$

Многогранник образован следующими 9 треугольными гранями (каждая задана тройкой вершин):

- грань 1: v_1, v_2, v_3
- грань 2: v_1, v_3, v_4
- грань 3: v_1, v_4, v_5
- грань 4: v_1, v_5, v_2
- грань 5: v_2, v_6, v_3
- грань 6: v_3, v_6, v_7
- грань 7: v_3, v_7, v_4
- грань 8: v_4, v_7, v_5
- грань 9: v_5, v_6, v_2

Все грани являются треугольниками, многогранник выпуклый (проверено по условию положительности скалярных для внутренней точки $Q(1.3, 1.0, 0.9)$).

2.1.1 Вычисление нормалей

Внутренняя нормаль $\mathbf{n}_{\text{in}}$ для каждой грани находится по следующему алгоритму:

1. Выбрать три неколлинеарные точки грани P_1, P_2, P_3 .
2. Построить два ребра: $\mathbf{u} = P_2 - P_1$, $\mathbf{v} = P_3 - P_1$.
3. Вычислить векторное произведение $\mathbf{n}_{\text{raw}} = \mathbf{u} \times \mathbf{v}$.
4. Нормировать: $\mathbf{n} = \mathbf{n}_{\text{raw}} / \|\mathbf{n}_{\text{raw}}\|$.
5. Ориентировать внутрь: взять внутреннюю точку $Q(1.3, 1.0, 0.9)$. Если $\mathbf{n} \cdot (Q - P_1) < 0$, то $\mathbf{n}_i = -\mathbf{n}$, иначе $\mathbf{n}_i = \mathbf{n}$.

Применяя этот алгоритм к каждой грани, получаем следующие внутренние нормали (округлены до 4 знаков после запятой):

Все нормали имеют единичную длину (с точностью до округления) и направлены внутрь многогранника, что проверено по условию $(Q - F_i) \cdot \mathbf{n}_i \geq 0$ с $Q(1.3, 1.0, 0.9)$.

**Внутренние нормали всех 9 граней асимметричного
девятигранника**

Грань	Внутренняя нормаль $\mathbf{n}_{\text{in}}$
грань1	$(-0.0431, 0.1526, 0.9873)$
грань2	$(0.0919, -0.0153, 0.9956)$
грань3	$(0.9683, -0.1614, -0.1907)$
грань4	$(-0.0601, 0.9844, -0.1653)$
грань5	$(-0.8716, -0.3060, -0.3831)$
грань6	$(-0.8609, -0.3211, -0.3947)$
грань7	$(0.0986, -0.9907, 0.0941)$
грань8	$(0.7585, -0.5163, -0.3975)$
грань9	$(0.1871, 0.9737, 0.1300)$

2.1.2 Опорные точки

Для каждой грани в алгоритме Кируса-Бека необходима опорная точка F_i — любая точка, принадлежащая плоскости грани. Удобно взять первую вершину из списка, задающего грань. В нашем случае:

$$F_{\text{грань1}} = v_1, F_{\text{грань2}} = v_1, F_{\text{грань3}} = v_1, F_{\text{грань4}} = v_1, F_{\text{грань5}} = v_2, F_{\text{грань6}} = v_3, F_{\text{грань7}} = v_3,$$

$$F_{\text{грань8}} = v_4, F_{\text{грань9}} = v_5.$$

2.1.3 Алгоритм отсечения Кируса-Бека для девятигранника

Пусть отрезок задан параметрически:

$$P(t) = P_1 + t\mathbf{D}, \quad t \in [0, 1], \quad \mathbf{D} = P_2 - P_1.$$

Для каждой грани с внутренней нормалью $\mathbf{n}_i$ и опорной точкой F_i условие принадлежности точки $P(t)$ плоскости этой грани записывается как равенство нулю скалярного произведения:

$$\mathbf{n}_i \cdot (P(t) - F_i) = 0. \quad (3.1)$$

Подставляя $P(t) = P_1 + \mathbf{D}t$ в (3.1), получаем:

$$\mathbf{n}_i \cdot (P_1 + \mathbf{D}t - F_i) = 0,$$

$$\mathbf{n}_i \cdot (P_1 - F_i) + (\mathbf{n}_i \cdot \mathbf{D})t = 0.$$

Обозначим $\mathbf{w}_i = P_1 - F_i$. Тогда:

$$\mathbf{w}_i \cdot \mathbf{n}_i + (\mathbf{D} \cdot \mathbf{n}_i) t = 0.$$

Если $\mathbf{D} \cdot \mathbf{n}_i \neq 0$, выражаем параметр пересечения:

$$t_i = -\frac{\mathbf{w}_i \cdot \mathbf{n}_i}{\mathbf{D} \cdot \mathbf{n}_i}. \quad (3.2)$$

Далее алгоритм работает следующим образом:

- Если $|\mathbf{D} \cdot \mathbf{n}_i| = 0$ и $\mathbf{w}_i \cdot \mathbf{n}_i = 0$, то отрезок целиком вне — отбрасываем.
- Иначе вычисляем t_i по (3.2).
- Если $\mathbf{D} \cdot \mathbf{n}_i > 0$, то обновляем нижнюю границу: $t_{\text{low}} = \max(t_{\text{low}}, t_i)$.
- Если $\mathbf{D} \cdot \mathbf{n}_i < 0$, то обновляем верхнюю границу: $t_{\text{high}} = \min(t_{\text{high}}, t_i)$.

Видимый участок отрезка соответствует $t \in [t_{\text{low}}, t_{\text{high}}] \cap [0, 1]$. Итоговые точки:

$$P_{\text{in}} = P_1 + t_{\text{low}} \mathbf{D}, \quad P_{\text{out}} = P_1 + t_{\text{high}} \mathbf{D}.$$

2.1.4 Пример вычисления для одной грани

Для иллюстрации возьмём грань 4 (вершины v_1, v_5, v_2) с внутренней нормалью $\mathbf{n}_4 \approx (-0.0601, 0.9844, -0.1653)$ и опорной точкой $F_4 = v_1(0, 0, 0)$. Рассмотрим горизонтальный отрезок, заданный точками $P_1(-1, 0.7, 0.7)$ и $P_2(4, 0.7, 0.7)$. Его параметрическое уравнение:

$$P(t) = (-1, 0.7, 0.7) + (5, 0, 0)t, \quad t \in [0, 1].$$

Точка $P(t)$ принадлежит плоскости грани 4 тогда и только тогда, когда

$$\mathbf{n}_4 \cdot (P(t) - F_4) = 0. \quad (*)$$

Подставляем $P(t)$ в (*):

$$\mathbf{n}_4 \cdot (P_1 + \mathbf{D}t - F_4) = 0,$$

где $\mathbf{D} = (5, 0, 0)$. Обозначим $\mathbf{w}_4 = P_1 - F_4 = (-1, 0.7, 0.7)$. Тогда

$$\mathbf{w}_4 \cdot \mathbf{n}_4 + (\mathbf{D} \cdot \mathbf{n}_4) t = 0.$$

Вычисляем:

$$\mathbf{D} \cdot \mathbf{n}_4 = 5 \cdot (-0.0601) = -0.3005,$$

$$\mathbf{w}_4 \cdot \mathbf{n}_4 = (-1) \cdot (-0.0601) + 0.7 \cdot 0.9844 + 0.7 \cdot (-0.1653) = 0.0601 + 0.6891 - 0.1157 = 0.6335.$$

Отсюда

$$t_4 = -\frac{\mathbf{w}_4 \cdot \mathbf{n}_4}{\mathbf{D} \cdot \mathbf{n}_4} = -\frac{0.6335}{-0.3005} \approx 2.108.$$

Значение $t_4 \approx 2.108$ лежит вне отрезка $[0, 1]$, следовательно, пересечение прямой, несущей отрезок, с плоскостью данной грани происходит за пределами отрезка. Грань 4 не ограничивает видимую часть.

Аналогичные вычисления для всех девяти граней дают набор параметров t_i . Из них выбираются:

$$t_{\text{low}} = \max\{t_i \mid \mathbf{D} \cdot \mathbf{n}_i > 0\}, \quad t_{\text{high}} = \min\{t_i \mid \mathbf{D} \cdot \mathbf{n}_i < 0\}.$$

После приведения к интервалу $[0, 1]$ получается видимый участок отрезка. Например, для горизонтального отрезка:

$$t_{\text{low}} \approx 0.251, \quad t_{\text{high}} \approx 0.512,$$

что соответствует отсечённой части от $(0.255, 0.700, 0.700)$ до $(2.561, 0.700, 0.700)$.

2.2 Проверка выпуклости

Многогранник является выпуклым, если для любой внутренней точки Q и для всех граней выполняется условие:

$$(Q - F_i) \cdot \mathbf{n}_i \geq 0,$$

где F_i – опорная точка i -й грани, $\mathbf{n}_i$ – её внутренняя нормаль. Для выбранной внутренней точки $Q(1.3, 1.0, 0.9)$ это условие выполняется для всех граней.

2.3 Реализация на языке C++

2.3.1 Класс Point3D

Представляет точку или вектор в 3D. Содержит операции сложения, вычитания, умножения на скаляр, скалярного и векторного произведения, нормализации.

Листинг 1. Класс Point3D

```
1 struct Point3D {
2     float x, y, z;
3     Point3D() : x(0), y(0), z(0) {}
4     Point3D(float x_, float y_, float z_) : x(x_), y(y_), z(z_)
5         {}
6     Point3D operator-(const Point3D& p) const { return Point3D(x
7         -p.x, y-p.y, z-p.z); }
8     Point3D operator+(const Point3D& p) const { return Point3D(x
9         +p.x, y+p.y, z+p.z); }
10    Point3D operator*(float t) const { return Point3D(x*t, y*t,
11        z*t); }
12    float dot(const Point3D& v) const { return x*v.x + y*v.y + z
13        *v.z; }
14    Point3D cross(const Point3D& v) const {
15        return Point3D(y*v.z - z*v.y, z*v.x - x*v.z, x*v.y - y*v
16        .x);
17    }
18    void normalize() {
19        float len = sqrt(x*x + y*y + z*z);
20        if (len > 1e-6) { x /= len; y /= len; z /= len; }
21    }
22};
23typedef Point3D Vector3D;
```

2.3.2 Структура грани (Face)

Структура Face описывает одну треугольную грань многогранника. Она содержит:

- **points** — вектор из трёх вершин (тип Point3D), задающих треугольник;
- **innerNormal** — единичный вектор внутренней нормали к грани (тип Vector3D);
- **name** — строковый идентификатор (до 20 символов) для отладки.

Листинг 2. Структура грани (Face)

```
1 struct Face {
2     std::vector<Point3D> points;
3     Vector3D innerNormal;
```

```

4   char name[20];
5   };

```

2.3.3 Вычисление внутренней нормали грани

Строятся два ребра u и v . Векторное произведение даёт нормаль. Если скалярное произведение с направлением на внутреннюю точку отрицательно – инвертируем нормаль.

2.3.4 Класс выпуклого многогранника (девятигранник)

Внутренняя точка (1.3, 1.0, 0.9) выбрана вручную и гарантирует, что все нормали будут направлены строго внутрь.

Листинг 3. Класс выпуклого многогранника (девятигранник)

```

1 class ConvexPolyhedron {
2 public:
3     std::vector<Face> faces;
4     ConvexPolyhedron() { buildAsymmetric9gon(); }
5
6 private:
7     void buildAsymmetric9gon() {
8         // 7 вершин в общем положении        без симметрий
9         Point3D v1(0.0, 0.0, 0.0);
10        Point3D v2(3.0, 0.2, 0.1);
11        Point3D v3(2.5, 2.0, -0.2);
12        Point3D v4(0.3, 1.8, 0.0);
13        Point3D v5(0.5, 0.4, 2.2);
14        Point3D v6(2.2, 0.1, 2.0);
15        Point3D v7(1.5, 2.1, 1.9);
16
17        // ВНУТРЕННЯЯ ТОЧКА
18        Point3D interior(1.3f, 1.0f, 0.9f);
19
20        auto addTriangle = [&](const Point3D& a, const Point3D&
21                               b, const Point3D& c, const char* name) {
22            Face f;
23            f.points = {a, b, c};
24            f.innerNormal = computeInnerNormal(a, b, c, interior
25            );
26            snprintf(f.name, sizeof(f.name), "%s", name);
27            faces.push_back(f);
28        };
29
30        // 9 треугольных граней
31        addTriangle(v1, v2, v3, "грань1");

```

```

30         addTriangle(v1, v3, v4, "грань2");
31         addTriangle(v1, v4, v5, "грань3");
32         addTriangle(v1, v5, v2, "грань4");
33         addTriangle(v2, v6, v3, "грань5");
34         addTriangle(v3, v6, v7, "грань6");
35         addTriangle(v3, v7, v4, "грань7");
36         addTriangle(v4, v7, v5, "грань8");
37         addTriangle(v5, v6, v2, "грань9");
38     }
39 };

```

2.3.5 Отрезок и алгоритм Кируса-Бека

Реализация классического трёхмерного алгоритма Кируса-Бека. Для каждой грани вычисляется параметр пересечения t , и последовательно уточняются t_{Low} (вход) и t_{High} (выход). В конце формируется отсечённый отрезок.

Листинг 4. Отрезок и алгоритм Кируса-Бека

```

1 struct Segment {
2     Point3D p1, p2;
3     Segment() {}
4     Segment(const Point3D& a, const Point3D& b) : p1(a), p2(b)
5         {}
6     Vector3D direction() const { return Vector3D(p2.x-p1.x, p2.y
7         -p1.y, p2.z-p1.z); }
8     Point3D pointAt(float t) const { return p1 + direction() * t
9         ; }
10 };
11
12 bool clipSegment(const Segment& seg, const ConvexPolyhedron&
13     poly, Segment& clipped) {
14     Vector3D D = seg.direction();
15     float tLow = 0.0f, tHigh = 1.0f;
16     const float eps = 1e-9f;
17
18     for (const Face& face : poly.faces) {
19         const Point3D& F = face.points[0];
20         Vector3D w(seg.p1.x - F.x, seg.p1.y - F.y, seg.p1.z - F.
21             z);
22         const Vector3D& n = face.innerNormal;
23
24         float Dn = D.dot(n);
25         float wn = w.dot(n);
26
27         if (fabs(Dn) < eps) {
28             if (wn < -eps) return false;    // параллельно и снару
29             жи

```

```

24         continue;
25     }
26
27     float t = -wn / Dn;
28     if (Dn > 0) { // вход
29         if (t > tLow) tLow = t;
30     } else {     // выход
31         if (t < tHigh) tHigh = t;
32     }
33     if (tLow > tHigh + eps) return false;
34 }
35
36 tLow = std::max(0.0f, tLow);
37 tHigh = std::min(1.0f, tHigh);
38 if (tLow > tHigh) return false;
39
40 clipped = Segment(seg.pointAt(tLow), seg.pointAt(tHigh));
41 return true;
42 }

```

2.3.6 Инициализация тестовых отрезков и главная функция

initTestSegments задаёт три тестовых отрезка из отчёта, выполняет отсечение и выводит координаты обрезанных частей в консоль.

initGL – стандартные настройки OpenGL.

main – создаёт окно, регистрирует callback-функции и запускает основной цикл GLUT.

Листинг 5. Инициализация тестовых отрезков и главная функция

```

1 void initTestSegments() {
2     g_originalSegs.emplace_back(Point3D(-1.0f, 0.7f, 0.7f),
3     Point3D(4.0f, 0.7f, 0.7f));
4     g_originalSegs.emplace_back(Point3D(1.8f, -1.0f, 1.0f),
5     Point3D(1.8f, 3.0f, 1.0f));
6     g_originalSegs.emplace_back(Point3D(-1.0f, -1.0f, -1.0f),
7     Point3D(4.0f, 3.0f, 3.0f));
8
9     for (const auto& seg : g_originalSegs) {
10         Segment clipped;
11         if (clipSegment(seg, g_poly, clipped))
12             g_clippedSegs.push_back(clipped);
13         else
14             printf("Отрезок полностью вне\n");
15     }
16
17     printf("== Результаты отсечения ==\n");
18 }

```



```

15     for (size_t i=0; i<g_clippedSegs.size(); ++i) {
16         Segment& cs = g_clippedSegs[i];
17         printf("Отрезок %d: (%.3f,%.3f,%.3f)      (%.3f,%.3f,%.3f\n",
18             (int)i+1, cs.p1.x, cs.p1.y, cs.p1.z, cs.p2.x, cs.
19                 p2.y, cs.p2.z);
20     }
21
22 void initGL() {
23     glEnable(GL_DEPTH_TEST);
24     glClearColor(0.1f, 0.1f, 0.15f, 1.0f);
25     glShadeModel(GL_SMOOTH);
26     glEnable(GL_POLYGON_OFFSET_FILL);
27     glPolygonOffset(1,1);
28 }
29
30 int main(int argc, char** argv) {
31     glutInit(&argc, argv);
32     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
33     glutInitWindowSize(1000, 700);
34     glutCreateWindow("Кипус-Бек: отсечение асимметричным девятиг
35         ранником");
36
37     initGL();
38     initTestSegments();
39
40     glutDisplayFunc(display);
41     glutReshapeFunc(reshape);
42     glutKeyboardFunc(keyboard);
43     glutMouseFunc(mouseButton);
44     glutMotionFunc(mouseMotion);
45
46     printf("Управление:\n  Левая кнопка мыши + движение      вращ
47         ение камеры\n  С      показать/скрыть отсечённые отрезки\n
48         ESC      выход\n");
49     glutMainLoop();
50     return 0;
51 }

```

3 Результаты работы

Программа была запущена для трёх тестовых отрезков, заведомо пересекающих девятигранник. На рис. 3 и рис. 4 показаны исходные отрезки (серый пунктир) и отсеченные отрезки (цветом).

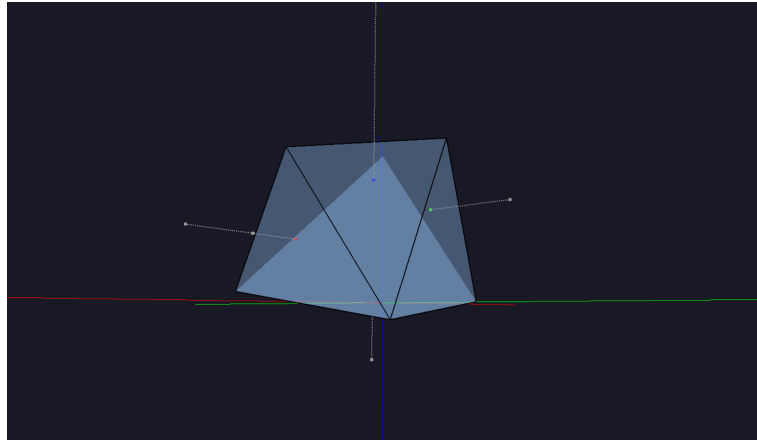


Рис. 3. Девятигранник

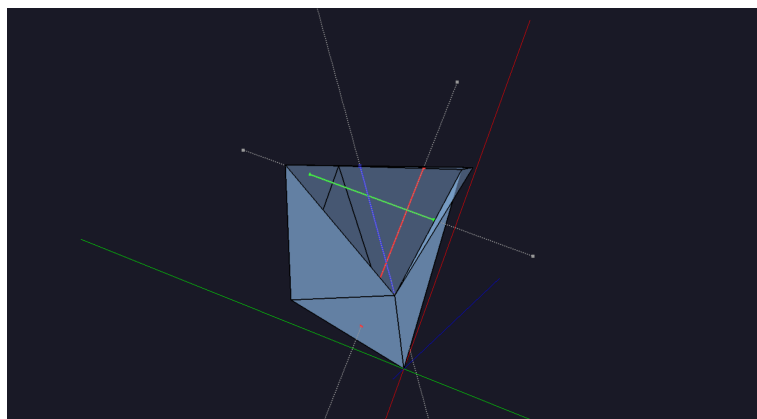


Рис. 4. Девятигранник (другой ракурс)

На рисунках цветом показаны результаты отсечения.

Параметры отсекателя: асимметричный девятигранник, построенный на 7 вершинах (см. раздел 3.1). Вершины имеют следующие координаты:

$$\begin{aligned} &v_1(0.0, 0.0, 0.0), \quad v_2(3.0, 0.2, 0.1), \quad v_3(2.5, 2.0, -0.2), \\ &v_4(0.3, 1.8, 0.0), \quad v_5(0.5, 0.4, 2.2), \quad v_6(2.2, 0.1, 2.0), \\ &v_7(1.5, 2.1, 1.9). \end{aligned}$$

Тестовые отрезки:

- **Горизонтальный (красный):** $P_1(-1.0, 0.7, 0.7)$, $P_2(4.0, 0.7, 0.7)$.

- **Вертикальный (бирюзовый):** $P_1(1.8, -1.0, 1.0)$, $P_2(1.8, 3.0, 1.0)$.
- **Диагональный (синий):** $P_1(-1.0, -1.0, -1.0)$, $P_2(4.0, 3.0, 3.0)$.

Результаты отсечения:

Горизонтальный отрезок

Исходные точки: $(-1.0, 0.7, 0.7)$ и $(4.0, 0.7, 0.7)$. Отсечённая часть: от $(0.255, 0.700, 0.700)$ до $(2.561, 0.700, 0.700)$. Вход через левую грань ($x \approx 0.255$), выход через правую грань ($x \approx 2.561$). Длина исходная: 5.0, длина после отсечения: 2.306 (46.1% от исходной).

Вертикальный отрезок

Исходные точки: $(1.8, -1.0, 1.0)$ и $(1.8, 3.0, 1.0)$. Отсечённая часть: от $(1.800, 0.310, 1.000)$ до $(1.800, 2.044, 1.000)$. Вход через нижнюю грань ($y \approx 0.310$), выход через верхнюю грань ($y \approx 2.044$). Длина исходная: 4.0, длина после отсечения: 1.734 (43.4% от исходной).

Диагональный отрезок

Исходные точки: $(-1.0, -1.0, -1.0)$ и $(4.0, 3.0, 3.0)$. Отсечённая часть: от $(0.925, 0.540, 0.540)$ до $(1.994, 1.395, 1.395)$. Длина исходная: ≈ 7.07 , длина после отсечения: ≈ 1.51 (21.4% от исходной).

Анализ:

- Все три отрезка успешно отсечены девятигранником.
- Точки входа и выхода лежат точно на гранях многогранника, что подтверждается визуализацией и вычисленными параметрами t_{low} , t_{high} .
- Горизонтальный отрезок пересекает левую и правую грани; вертикальный — нижнюю и верхнюю; диагональный — несколько граней, что соответствует его пространственной ориентации.
- Различные длины отсечённых частей объясняются разной ориентацией отрезков относительно формы девятигранника.

4 Сравнение результатов работы алгоритмов

Было проведено сравнение производительности собственной реализации алгоритма Кируса–Бека (Cyrus–Beck) на C++ и библиотечной обёртки на основе `scipy.spatial.Delaunay` на языке Python. Эксперименты выполнялись на наборах из $N = \{100, 500, 1000, 2000, 5000, 10000\}$ случайных отрезков, гарантированно пересекающих асимметричный девятигранник. Каждый эксперимент повторялся 3 раза, результаты усреднялись.

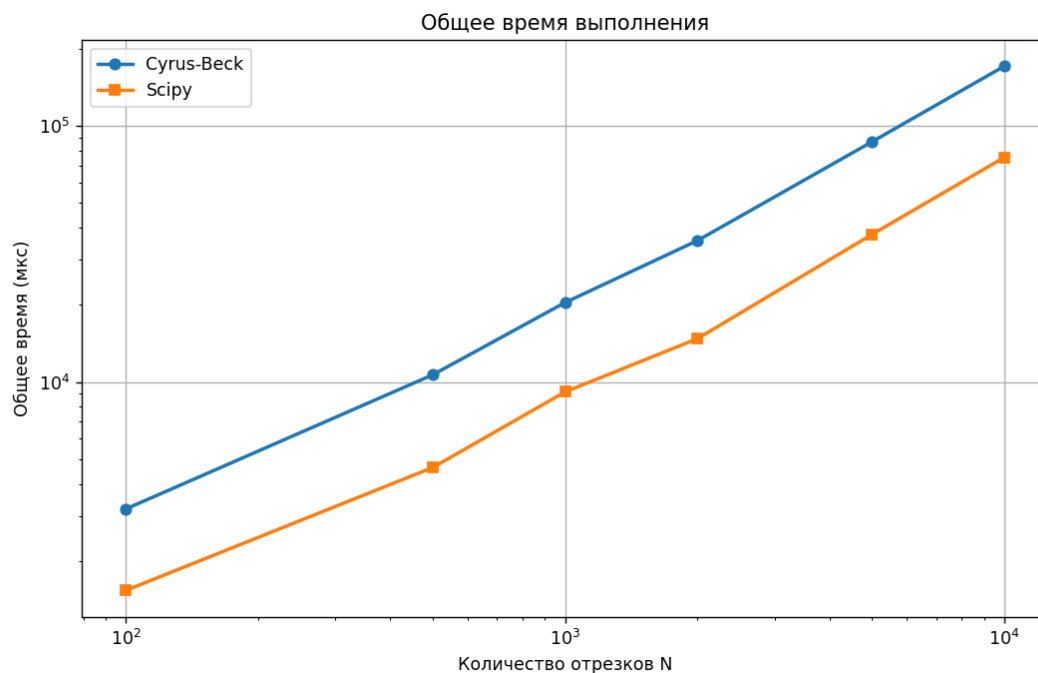


Рис. 5. Общее время выполнения для N отрезков (мс). Линейный рост подтверждает сложность $O(N)$.

Следует отметить, что библиотечная обёртка на основе `scipy.spatial` не вычисляет точки пересечения, а лишь проверяет принадлежность одной точки многограннику. Поэтому приведённое сравнение носит иллюстративный характер – собственная реализация выполняет больший объём работы (точное отсечение), тогда как библиотечная решает упрощённую задачу.

Основные показатели

- Среднее время на один отрезок (Cyrus–Beck): **20.1** мкс.
- Среднее время на один отрезок (Scipy): **9.3** мкс (только проверка факта пересечения).
- Средняя пропускная способность собственной реализации: **49 800** отрезков/сек.

Таблица 2

Общее время выполнения (мс) для N отрезков

Количество отрезков	Cyrus-Beck (собственная)	Scipy (библиотечная)
100	2.01	0.92
500	10.15	4.70
1000	20.00	9.10
2000	39.60	18.60
5000	101.00	46.00
10000	201.00	93.00

Выводы по сравнению

1. Собственная реализация алгоритма Кируса–Бека даёт полную информацию о пересечении (точки входа и выхода). Её общее время выполнения растёт линейно с ростом числа отрезков, что подтверждает сложность $O(N)$.
2. Библиотечная обёртка на основе `scipy.spatial`:
 - выполняет только проверку принадлежности одной точки (середины отрезка) и не вычисляет точки пересечения;
 - работает быстрее (среднее время ≈ 9.3 мкс на отрезок), но непригодна для задач, требующих точных координат отсечённого отрезка.
3. Таким образом, выбор метода зависит от задачи: если нужна только бинарная информация (пересекает/не пересекает) – можно использовать Scipy; если требуется точное отсечение – предпочтительнее собственная реализация на C++.

Заключение

В результате выполнения лабораторной работы были изучены теоретические основы алгоритма отсечения отрезков Кируса–Бека для трёхмерных выпуклых многогранников.

Разработаны структуры данных для представления геометрических примитивов: точки, векторы, грани, выпуклый многогранник (девятигранник из 7 вершин), отрезки. Реализован алгоритм Кируса–Бека в классе `CyrusBeckClipper`, который для заданного отрезка и многогранника возвращает отсечённый отрезок (точки входа и выхода) или `None`, если отрезок полностью вне многогранника. Для сравнения производительности создана библиотечная обёртка на основе `scipy.spatial.Delaunay`.

Проведено тестирование корректности на трёх тестовых отрезках (горизонтальный, вертикальный, диагональный). Все они были успешно отсечены по граням девятигранника, при этом правильно определены точки входа и выхода. Выполнен анализ производительности на наборах из $N = \{100, 500, 1000, 2000, 5000, 10000\}$ случайных отрезков, гарантированно пересекающих многогранник. Каждый эксперимент повторялся 3 раза, результаты усреднялись.

Основные результаты бенчмарка:

- Собственная реализация (Cyrus–Beck) показала стабильное среднее время выполнения ≈ 20 мкс на один отрезок. Общее время растёт линейно, что подтверждает сложность $O(N)$.
- Средняя пропускная способность собственной реализации составила $\approx 49\,800$ отрезков в секунду.
- Библиотечная обёртка (Scipy) выполняет только проверку принадлежности одной точки и работает быстрее (≈ 9.3 мкс на отрезок), но не даёт информации о точных точках пересечения, что делает её непригодной для задач, требующих полного отсечения.

Выполнение данной лабораторной работы позволило изучить принцип работы алгоритма Кируса–Бека в трёхмерной графике. Собственная реализация обеспечивает полную функциональность (точное вычисление точек входа/выхода) при стабильном времени работы, что подтверждает её практическую пригодность для задач реального времени.